Genomic Workflows and Cloud Computing

Processing genomic DNA

- Lots of steps
  - Trimming, FASTQC, Alignment, Sorting, Deduplication, Alignment QC . . .
- Copy/paste is slow and error-prone
- Changing variable names (don't miss any!)
- Waiting for each step to finish

Wrapping in a bash script

```
#!/bin/bash

SAMPLE=$1
FASTQ1_PATH=$2
FASTQ2_PATH=$3
REFERENCE=$4
RGID=$5
OUTPATH=$6

/usr/gitc/bwa mem -R "@RG\tID:$RGID\tPL:ILLUMINA\SM:$SAMPLE" $REFERENCE
$FASTQ1_PATH $FASTQ2_PATH | /usr/local/bin/samtools view -1 -o
$OUTPATH/aligned.bam -

samtools sort $OUTPATH/aligned.bam >$OUTPATH/aligned_sorted.bam
rm $OUTPATH/aligned.bam

. . .
```

Wrapping in a bash script

```
#!/bin/bash

SAMPLE=$1
FASTQ1_PATH=$2
FASTQ2_PATH=$3
REFERENCE=$4
RGID=$5
OUTPATH=$6

/usr/gitc/bwa mem -R "@RG\tID:$RGID\tPL:ILLUMINA\SM:$SAMPLE" $REFERENCE $FASTQ1_PATH
$FASTQ2_PATH | /usr/local/bin/samtools view -1 -o $OUTPATH/aligned.bam -

samtools sort $OUTPATH/aligned.bam >$OUTPATH/aligned_sorted.bam
rm $OUTPATH/aligned.bam

. . .
```

This works okay for a little while

Wrapping in a bash script

```
#!/bin/bash

SAMPLE=$1
FASTQ1_PATH=$2
FASTQ2_PATH=$3
REFERENCE=$4
RGID=$5
OUTPATH=$6

/usr/gitc/bwa mem -R "@RG\tID:$RGID\tPL:ILLUMINA\SM:$SAMPLE" $REFERENCE $FASTQ1_PATH
$FASTQ2_PATH | /usr/local/bin/samtools view -1 -o $OUTPATH/aligned.bam -

samtools sort $OUTPATH/aligned.bam >$OUTPATH/aligned_sorted.bam
rm $OUTPATH/aligned.bam

. . .
```

This works okay for a little while

- which environment are you running this in? (what tool versions, etc?)

Wrapping in a bash script

```
#!/bin/bash

SAMPLE=$1
FASTQ1_PATH=$2
FASTQ2_PATH=$3
REFERENCE=$4
RGID=$5
OUTPATH=$6

/usr/gitc/bwa mem -R "@RG\tID:$RGID\tPL:ILLUMINA\SM:$SAMPLE" $REFERENCE $FASTQ1_PATH
$FASTQ2_PATH | /usr/local/bin/samtools view -1 -o $OUTPATH/aligned.bam -

samtools sort $OUTPATH/aligned.bam >$OUTPATH/aligned_sorted.bam
rm $OUTPATH/aligned.bam

. . .
```

This works okay for a little while

- which environment are you running this in? (what tool versions, etc?)
- what happens when a command dies halfway through your script?

Wrapping in a bash script

```
#!/bin/bash

SAMPLE=$1
FASTQ1_PATH=$2
FASTQ2_PATH=$3
REFERENCE=$4
RGID=$5
OUTPATH=$6

/usr/gitc/bwa mem -R "@RG\tID:$RGID\tPL:ILLUMINA\SM:$SAMPLE" $REFERENCE $FASTQ1_PATH
$FASTQ2_PATH | /usr/local/bin/samtools view -1 -o $OUTPATH/aligned.bam -

samtools sort $OUTPATH/aligned.bam >$OUTPATH/aligned_sorted.bam
rm $OUTPATH/aligned.bam

. . .
```

This works okay for a little while

- which environment are you running this in? (what tool versions, etc?)
- what happens when a command dies halfway through your script?
- how do you handle intermediate files that pile up?

Wrapping in a bash script

```
#!/bin/bash

SAMPLE=$1
FASTQ1_PATH=$2
FASTQ2_PATH=$3
REFERENCE=$4
RGID=$5
OUTPATH=$6

/usr/gitc/bwa mem -R "@RG\tID:$RGID\tPL:ILLUMINA\SM:$SAMPLE" $REFERENCE $FASTQ1_PATH
$FASTQ2_PATH | /usr/local/bin/samtools view -1 -o $OUTPATH/aligned.bam -

samtools sort $OUTPATH/aligned.bam >$OUTPATH/aligned_sorted.bam
rm $OUTPATH/aligned.bam

. . .
```

This works okay for a little while

- which environment are you running this in? (what tool versions, etc?)
- what happens when a command dies halfway through your script?
- how do you handle intermediate files that pile up?
- how do you debug failures?

Wrapping in a bash script

```
#!/bin/bash

SAMPLE=$1
FASTQ1_PATH=$2
FASTQ2_PATH=$3
REFERENCE=$4
RGID=$5
OUTPATH=$6

/usr/gitc/bwa mem -R "@RG\tID:$RGID\tPL:ILLUMINA\SM:$SAMPLE" $REFERENCE $FASTQ1_PATH
$FASTQ2_PATH | /usr/local/bin/samtools view -1 -o $OUTPATH/aligned.bam -

samtools sort $OUTPATH/aligned.bam >$OUTPATH/aligned_sorted.bam
rm $OUTPATH/aligned.bam

. . .
```

This works okay for a little while

- which environment are you running this in? (what tool versions, etc?)
- what happens when a command dies halfway through your script?
- how do you handle intermediate files that pile up?
- how do you debug failures?
- what happens when you want to scale this up, and move to the cloud?

Workflows

- Come in many flavors (CWL, WDL, Nextflow, Snakemake)
- Solve many of these problems, plus some others

Anatomy of a workflow

- Base unit is individual steps with inputs and outputs
- Think of them as building blocks

Anatomy of a workflow

```
1   #!/usr/bin/env cwl-runner
2
3   cwlVersion: v1.0
4   class: CommandLineTool
5   label: "samtools flagstat"
6   baseCommand: ["/usr/local/bin/samtools", "flagstat"]
7   requirements:
8       - class: ResourceRequirement
9         ramMin: 4000
10      - class: DockerRequirement
11        dockerPull: "quay.io/biocontainers/samtools:1.11--h6270b1f_0"
12  stdout: "$(inputs.bam.basename).flagstat"
13  inputs:
14      bam:
15          type: File
16          inputBinding:
17              position: 1
18          secondaryFiles: [^.bai]
19  outputs:
20      flagstats:
21          type: stdout
```

Anatomy of a workflow

Description of the tool

```
#!/usr/bin/env cwl-runner

cwlVersion: v1.0
class: CommandLineTool
label: "samtools flagstat"
baseCommand: ["/usr/local/bin/samtools", "flagstat"]
requirements:
    - class: ResourceRequirement
      ramMin: 4000
    - class: DockerRequirement
      dockerPull: "quay.io/biocontainers/samtools:1.11--h6270b1f_0"
stdout: "$(inputs.bam.basename).flagstat"
inputs:
    bam:
        type: File
        inputBinding:
            position: 1
        secondaryFiles: [^.bai]
outputs:
    flagstats:
        type: stdout
```

Anatomy of a workflow

Description of the tool

how many resources are needed

```
#!/usr/bin/env cwl-runner

cwlVersion: v1.0
class: CommandLineTool
label: "samtools flagstat"
baseCommand: ["/usr/local/bin/samtools", "flagstat"]
requirements:
    - class: ResourceRequirement
      ramMin: 4000
    - class: DockerRequirement
      dockerPull: "quay.io/biocontainers/samtools:1.11--h6270b1f_0"
stdout: "$(inputs.bam.basename).flagstat"
inputs:
    bam:
        type: File
        inputBinding:
            position: 1
        secondaryFiles: [^.bai]
outputs:
    flagstats:
        type: stdout
```

Anatomy of a workflow

Description of the tool

how many resources are needed

Docker image to use

```
#!/usr/bin/env cwl-runner

cwlVersion: v1.0
class: CommandLineTool
label: "samtools flagstat"
baseCommand: ["/usr/local/bin/samtools", "flagstat"]
requirements:
    - class: ResourceRequirement
      ramMin: 4000
    - class: DockerRequirement
      dockerPull: "quay.io/biocontainers/samtools:1.11--h6270b1f_0"
stdout: "$(inputs.bam.basename).flagstat"
inputs:
    bam:
        type: File
        inputBinding:
            position: 1
        secondaryFiles: [^.bai]
outputs:
    flagstats:
        type: stdout
```

Anatomy of a workflow

Description of the tool

Main part of the command

how many resources are needed

Docker image to use

```
#!/usr/bin/env cwl-runner

cwlVersion: v1.0
class: CommandLineTool
label: "samtools flagstat"
baseCommand: ["/usr/local/bin/samtools", "flagstat"]
requirements:
    - class: ResourceRequirement
      ramMin: 4000
    - class: DockerRequirement
      dockerPull: "quay.io/biocontainers/samtools:1.11--h6270b1f_0"
stdout: "$(inputs.bam.basename).flagstat"
inputs:
    bam:
        type: File
        inputBinding:
            position: 1
        secondaryFiles: [^.bai]
outputs:
    flagstats:
        type: stdout
```

Anatomy of a workflow

Description of the tool

Main part of the command

how many resources are needed

Docker image to use

Inputs and details

```
#!/usr/bin/env cwl-runner

cwlVersion: v1.0
class: CommandLineTool
label: "samtools flagstat"
baseCommand: ["/usr/local/bin/samtools", "flagstat"]
requirements:
    - class: ResourceRequirement
      ramMin: 4000
    - class: DockerRequirement
      dockerPull: "quay.io/biocontainers/samtools:1.11--h6270b1f_0"
stdout: "$(inputs.bam.basename).flagstat"
inputs:
    bam:
        type: File
        inputBinding:
            position: 1
        secondaryFiles: [^.bai]
outputs:
    flagstats:
        type: stdout
```

Anatomy of a workflow

Description of the tool

Main part of the command

how many resources are needed

Docker image to use

Inputs and details

Outputs and details

```
#!/usr/bin/env cwl-runner

cwlVersion: v1.0
class: CommandLineTool
label: "samtools flagstat"
baseCommand: ["/usr/local/bin/samtools", "flagstat"]
requirements:
    - class: ResourceRequirement
      ramMin: 4000
    - class: DockerRequirement
      dockerPull: "quay.io/biocontainers/samtools:1.11--h6270b1f_0"
stdout: "$(inputs.bam.basename).flagstat"
inputs:
    bam:
        type: File
        inputBinding:
            position: 1
        secondaryFiles: [^.bai]
outputs:
    flagstats:
        type: stdout
```

Anatomy of a workflow

Description of the tool

Main part of the command

how many resources are needed

```
1   #!/usr/bin/env cwl-runner
2
3   cwlVersion: v1.0
4   class: CommandLineTool
5   label: "samtools flagstat"
6   baseCommand: ["/usr/local/bin/samtools", "flagstat"]
7   requirements:
8       - class: ResourceRequirement
9         ramMin: 4000
10      - class: DockerRequirement
```

This is a lot - why not just use:

```
samtools flagstat myreads.bam >myreads.bam.flagstat
```

Inputs and details

Outputs and details

```
15          type: File
16          inputBinding:
17              position: 1
18          secondaryFiles: [^.bai]
19  outputs:
20      flagstats:
21          type: stdout
```

Chaining tools together

```yaml
inputs:
    bam:
        type: File
        secondaryFiles: [^.bai, .bai]
    reference:
        type:
            - string
            - File
        secondaryFiles: [.fai, ^.dict]
```

```yaml
collect_wgs_metrics:
    run: ../tools/collect_wgs_metrics.cwl
    in:
        bam: bam
        reference: reference
    out:
        [wgs_metrics]
samtools_flagstat:
    run: ../tools/samtools_flagstat.cwl
    in:
        bam: bam
    out: [flagstats]
```

```yaml
collect_gc_bias_metrics:
    run: ../tools/collect_gc_bias_metrics.cwl
    in:
        bam: bam
        reference: reference
        metric_accumulation_level: picard_metric_accumulation_level
    out:
        [gc_bias_metrics, gc_bias_metrics_chart, gc_bias_metrics_summary]

cgpbigwig_bamcoverage:
    run: ../tools/bam_to_bigwig.cwl
    in:
        bam: bam
        reference: reference
    out:
        [outfile]
```

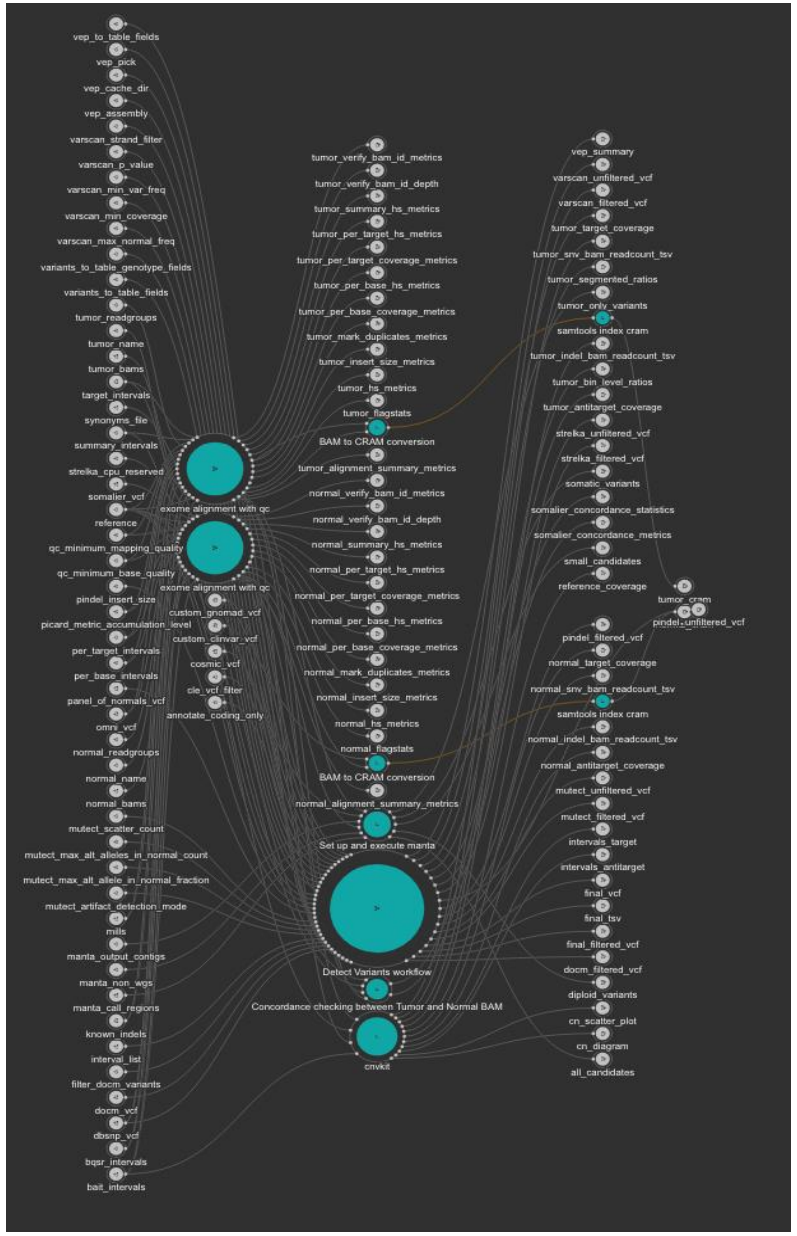
Building up workflows

hundreds of building blocks

add_vep_fields_to_table.cwl
align_and_tag.cwl
annotsv.cwl
apply_bqsr.cwl
bam_readcount.cwl
bam_to_bigwig.cwl
bam_to_cram.cwl
bam_to_fastq.cwl
bam_to_sam.cwl
bcftools_merge.cwl
bedgraph_to_bigwig.cwl
bedtools_intersect.cwl
bgzip.cwl
biscuit_align.cwl
biscuit_markdup.cwl
bqsr.cwl
call_duplex_consensus.cwl
cellmatch_lineage.cwl
cellranger_count.cwl
clip_overlap.cwl
cnvkit_batch.cwl
cnvnator.cwl
collect_alignment_summary_metrics.cwl
collect_gc_bias_metrics.cwl
extract_umis.cwl
fastq_to_bam.cwl
filter_consensus.cwl
filter_known_variants.cwl
hisat2_align.cwl
hla_consensus.cwl
homer_tag_directory.cwl
manta_somatic.cwl
mark_duplicates_and_sort.cwl
mark_illumina_adapters.cwl
merge_bams_samtools.cwl
. . .

Putting them together

Plumbing – inputs and outputs connected through pipes



An alignment/somatic mutation calling pipeline

- 400,000 lines of code in the `genome/analysis-workflows` github repository

- Dozens of data types and approaches
  - exome/WGS/targeted (somatic/germline)
  - bisulfite
  - RNAseq
  - single-cell (TCR, 5’/3’, ATAC)
  - RNAseq (expression, fusions, splicing)
  - ATAC/ChIPseq
  - etc

master · analysis-workflows / definitions / pipelines / · Go to file · Add file

johnegarza Merge pull request #988 from johnegarza/immuno_vcf_filter_updates · 77ec4f2 on Jan 21 · History

alignment_exome.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
alignment_exome_nonhuman.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
alignment_umi_duplex.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
alignment_umi_molecular.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
alignment_wgs.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
alignment_wgs_nonhuman.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
aml_trio_cle.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
aml_trio_cle_gathered.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
bisulfite.cwl	Changed the documentation to a doc line	2 years ago
chipseq.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
chipseq_alignment_nonhuman.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
detect_variants.cwl	Generalized parameter names and improved documentation	last month
detect_variants_nonhuman.cwl	Add doc line about scatter_count to pipelines.	5 months ago
detect_variants_wgs.cwl	Generalized parameter names and improved documentation	last month
downsample_and_recall.cwl	Removed variant_index variables as not used in GATK4	7 months ago
gathered_downsample_and_recall.cwl	Gathered downsample_and_recall to store gVCFs in per-BAM directories.	14 months ago
germline_exome.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
germline_exome_gvcf.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
germline_exome_hla_typing.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
germline_wgs.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
germline_wgs_gvcf.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
immuno.cwl	Merge pull request #988 from johnegarza/immuno_vcf_filter_updates	last month
rnaseq.cwl	update stringtie to accept indexed bam	last month
rnaseq_star_fusion.cwl	update stringtie to accept indexed bam	last month
rnaseq_star_fusion_with_xenosplit.cwl	made inputs for rna-xenosplit consistent with the other rna workflows	last month
somatic_exome.cwl	Merge pull request #988 from johnegarza/immuno_vcf_filter_updates	last month
somatic_exome_cle.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
somatic_exome_cle_gathered.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
somatic_exome_gathered.cwl	Merge pull request #988 from johnegarza/immuno_vcf_filter_updates	last month
somatic_exome_nonhuman.cwl	s/exmaple/example/	last month
somatic_wgs.cwl	Merge pull request #988 from johnegarza/immuno_vcf_filter_updates	last month
tumor_only_detect_variants.cwl	adding docm documentation	5 months ago
tumor_only_exome.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
tumor_only_wgs.cwl	Remove `i.e.`	last month

- 400,000 lines of code in the `genome/analysis-workflows` github repository

- Dozens of data types and approaches
  - exome/WGS
  - bisulfite
  - RNAseq
  - single-cell (TCR, 5'/3', ATAC)
  - RNAseq (expression, fusions, splicing)
  - ATAC/ChIPseq
  - etc

You don't have to write it from scratch

master / analysis-workflows / definitions / pipelines /

Go to file | Add file

johnegarza Merge pull request #988 from johnegarza/immuno_vcf_filter_updates ... 77ec4f2 on Jan 21 History

alignment_exome.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
alignment_exome_nonhuman.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
alignment_umi_duplex.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
alignment_umi_molecular.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
alignment_wgs.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
alignment_wgs_nonhuman.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
aml_trio_cle.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
aml_trio_cle_gathered.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
bisulfite.cwl	Changed the documentation to a doc line	2 years ago
chipseq.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
chipseq_alignment_nonhuman.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
detect_variants.cwl	Generalized parameter names and improved documentation	last month
		5 months ago
		last month
		7 months ago
		14 months ago
germline_exome.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
germline_exome_gvcf.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
germline_exome_hla_typing.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
germline_wgs.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
germline_wgs_gvcf.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
immuno.cwl	Merge pull request #988 from johnegarza/immuno_vcf_filter_updates	last month
rnaseq.cwl	update stringtie to accept indexed bam	last month
rnaseq_star_fusion.cwl	update stringtie to accept indexed bam	last month
rnaseq_star_fusion_with_xenosplit.cwl	made inputs for rna-xenosplit consistent with the other rna workflows	last month
somatic_exome.cwl	Merge pull request #988 from johnegarza/immuno_vcf_filter_updates	last month
somatic_exome_cle.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
somatic_exome_cle_gathered.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
somatic_exome_gathered.cwl	Merge pull request #988 from johnegarza/immuno_vcf_filter_updates	last month
somatic_exome_nonhuman.cwl	s/exmaple/example/	last month
somatic_wgs.cwl	Merge pull request #988 from johnegarza/immuno_vcf_filter_updates	last month
tumor_only_detect_variants.cwl	adding docm documentation	5 months ago
tumor_only_exome.cwl	Propagate documentation of sequence inputs to other pipeline CWLs.	last month
tumor_only_wgs.cwl	Remove `i.e.`	last month

Ideal situation

- You choose a workflow from a repository
- You loop through your samples, creating one input per case
- The workflow runs, providing progress and restarting when needed
- You get neatly organized folders full of identically processed results

Infrastructure independence

- Containers (like docker) + workflows = platform independence
  - run on your laptop
  - run on the cluster
  - run on the cloud
- Get the same results!
  - without having to install dozens or hundreds of tools on a server

Reproducibility

- Ideally, you could send someone a link to your data and the workflow, and they could sit down, run it, and reproduce the figures/tables in your paper exactly

- That's a high bar, but workflows get us a pretty long way down that road

- They're good for your sanity too:

  Hell is other people's data.
  Hell is also your own data 6 months later.

Code + Data

- Data sharing is the other piece of reproducibility

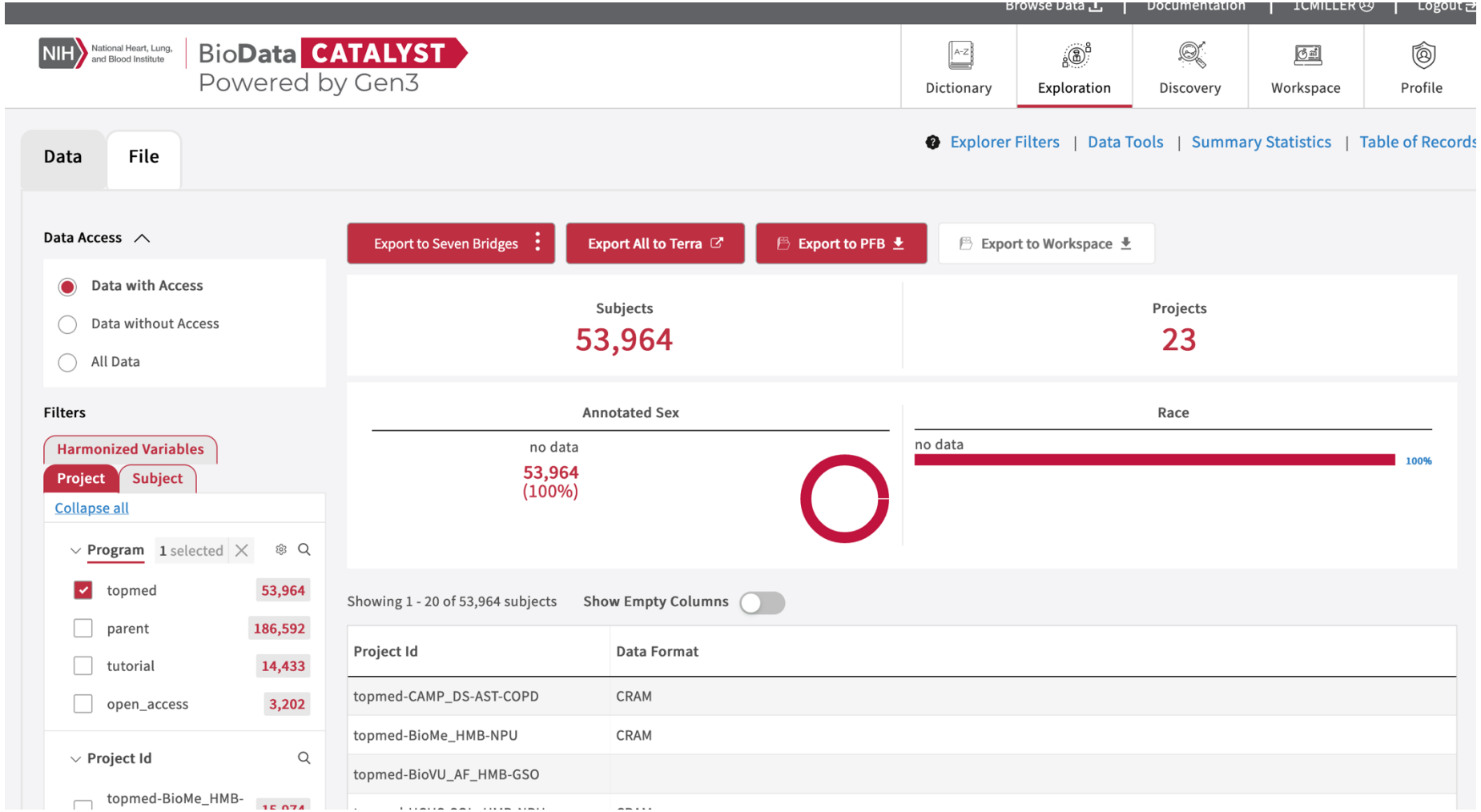
FAIR Principles of data:

- Findable
- Accessible
- Interoperable
- Reproducible

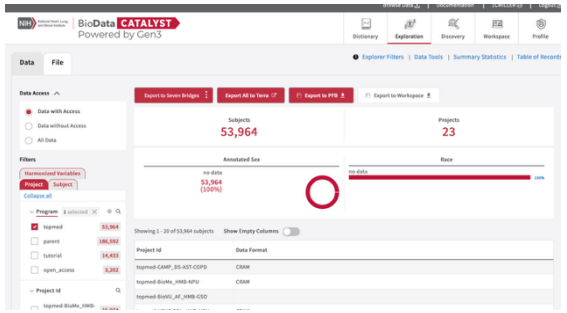
Examples of good places to deposit data

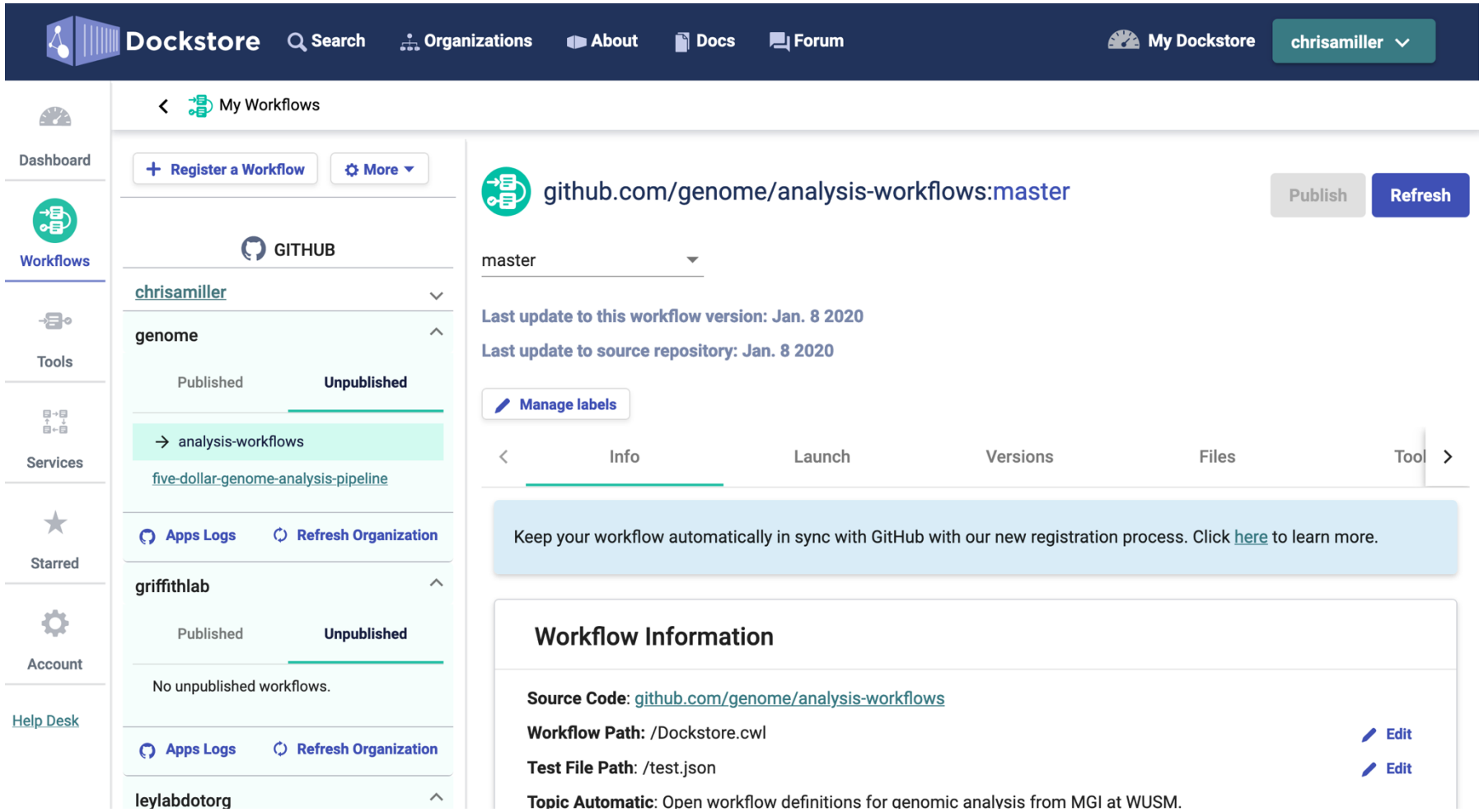
- NCBI repositories
  - SRA – Short Read Archive
  - dbGaP – front-end/access control for human data in SRA
  - GEO – rich metadata associated with experiments (RNA, scRNA, arrays, etc)
- Organism specific repos (Flybase, etc)
- General data repos that assign a doi (zenodo.org)
- Institutional repositories – your university library probably runs one
- **NOT** a lab website or a Google bucket (what happens in 5 years?)

What happens when this works well?

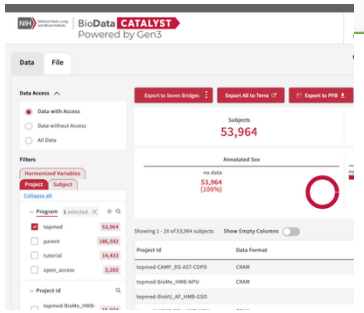


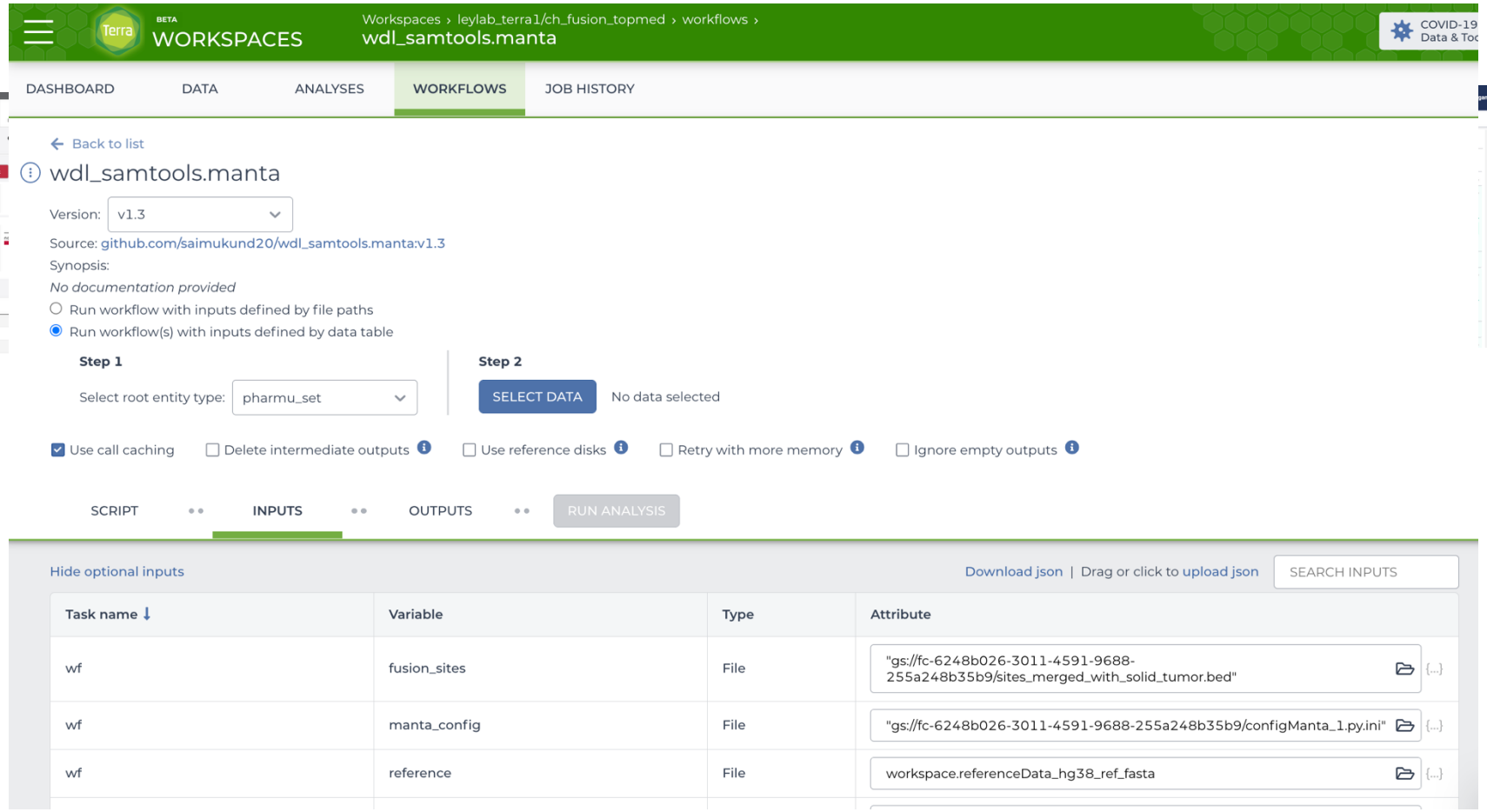
What happens when this works well?

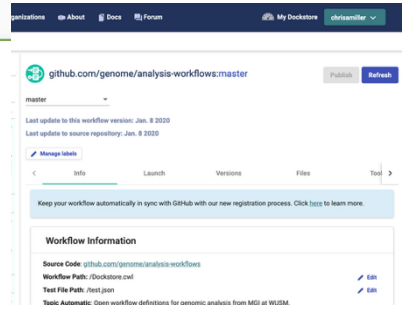




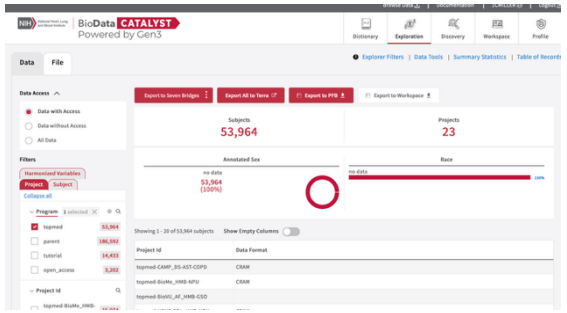
What happens when this works well?

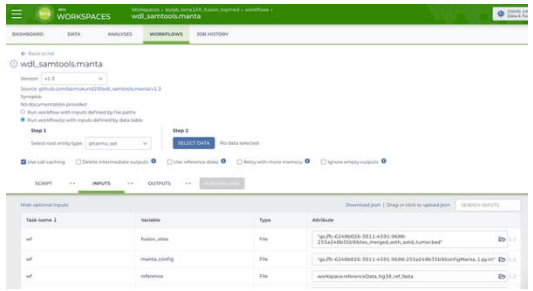


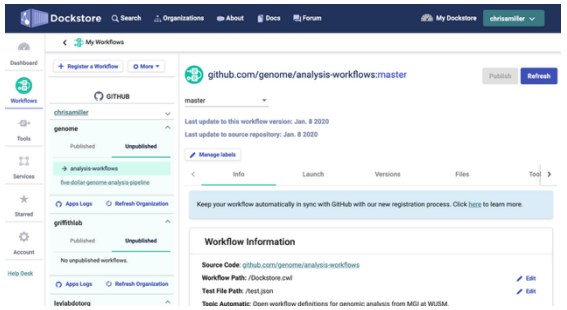




What happens when this works well?







Analysis results for >50,000 Whole Genome Samples in a few hours

Cloud Computing

Pairs really well with genomic workflows

Why go to the cloud?

- Performance & reliability
- Access to additional storage and compute as needed
- Greater control over compute environment
    - Ability to create persistent web services
- Data sharing needs
- Access to technologies not available on local WashU compute
- It's a valuable skill? It's the future?

Why not go to the cloud?

- Cost
- Complexity. Its another thing to learn
- Security?

Public Cloud & in-house compute

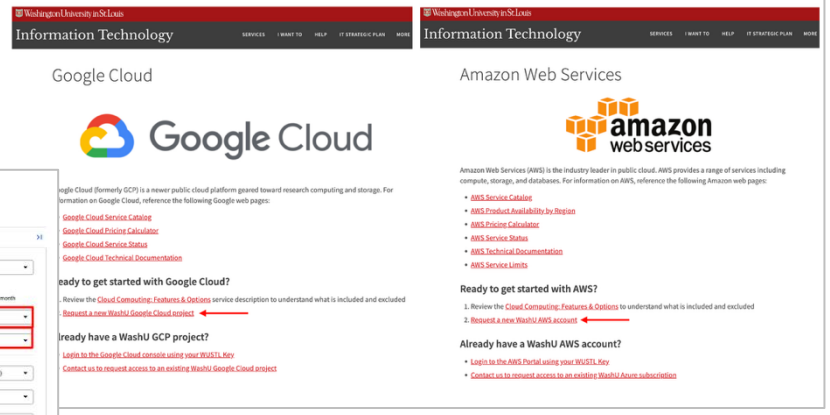
- Using the cloud is not the same as using the compute1 cluster
  - Not HPC-oriented (no LSF, à la `bsub`; no NFS-based file system)
  - Essence of scientific cloud computing:
    i. Create a virtual machine (VM)
    ii. Download ("localize") Input Data onto VM from Cloud Storage ("bucket")
    iii. Install software onto VM
    iv. Run computation(s)
    v. Upload Output Data ("de-localize") on the VM to Cloud Storage
    vi. Delete VM

- "research" on the cloud vs. local in-house compute
  - Exploratory compute on the cloud can get expensive very quick.
  - Try to do small-scale "research" on in-house compute, and then move to the cloud for large-scale "production" tasks
  - Use the cloud for functionality not supported by local compute
  - Use the cloud as a backup when local system is unstable

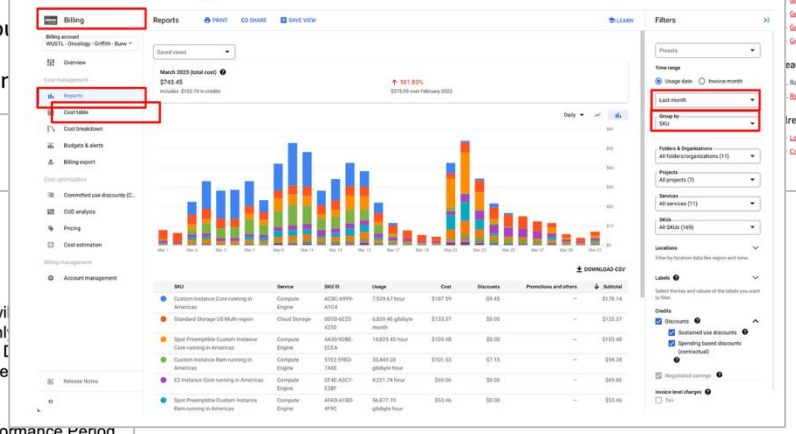
Getting started – the gory details

1. **Initial setup.** Request an AWS or GCP cloud account from WUIT
2. **POs.** Request a purchase order from your department and link to the account and encumber with appropriate sources of funding
3. **Login.** Test login and user permissions to access billing information and other necessary functionality. Request additional permissions from WUIT as needed.
4. **Users.** Request addition of users from the lab who need access. O to add them yourself using their WUSTL credentials.
5. **Invoices.** Set up routing of DLT (AWS) or Burwood (GCP) bills to you
6. **Alerts.** Set up billing dashboards and monthly billing alerts.
7. **Cost control.** Consider cost optimizations such as reserved instar

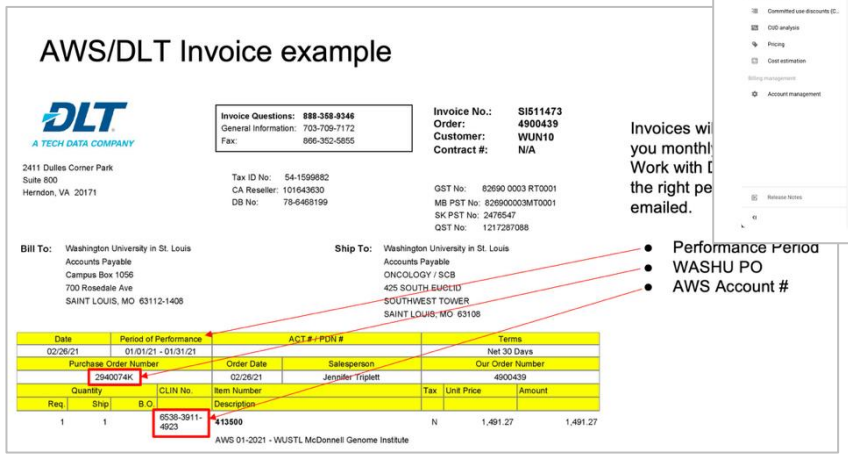
Project initiation – request a new GCP "project" or AWS "account"



Google Billing Dashboard



AWS/DLT Invoice example

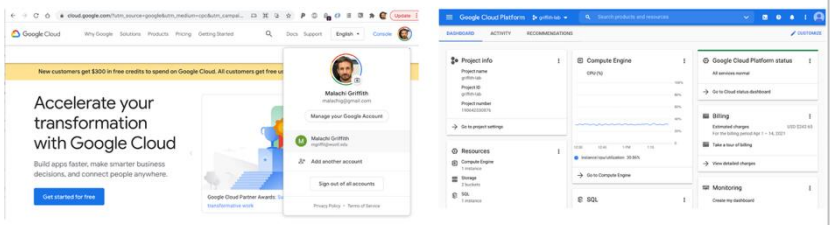


Invoices wi
you monthl
Work with D
the right pe
emailed.

- Performance Period
- WASHU PO
- AWS Account #

Logging into GCP (https://cloud.google.com/)

- Your WASHU email will function as a Google account
  - A very limited functionality Google account associated with your WASHU login
  - You now have two Google accounts. If you use the Google ecosystem a lot, this will be annoying.
  - Better functionality would happen with Google Workspace (formerly known as "G Suite") integration.
  - Use Google Chrome Profiles to keep the two accounts separate



https://github.com/genome/bfx-workshop/tree/master/archive/v2023-2024/lectures/week_22

Okay, I'm ready to actually do something on the cloud…

WUSTL Oncology

Division of Oncology, Washington University School of Medicine

United States of America | https://oncology.wustl.edu/ | Part of Washington University in St....

Overview | Repositories 6 | Projects | Packages | Teams 1 | People 12 | Security | Insights

README.md

Washington University School of Medicine in St. Louis

Code and documentation from the WUSTL Division of Oncology

These repositories represent accumulated tools and knowledge that enable labs to get up and running with cloud computing, mostly using cancer genome analysis as the use case. Some of the contents include:

- How to get your lab signed up for Google Cloud access through WUSTL IT
- End-to-end genomic workflows for variant calling, rnaseq analyses, epigenomics, and more.
- Guidance for running these pipelines (or others) on Google Cloud, including complete walkthrough examples of using it to run an immunogenomics pipeline from data you've already put on the cloud or from data stored on the local compute1 cluster
- Guides to running workflows on the local compute cluster, using GMS (example) for workflow orchestration or Cromwell
- Annotation files pre-loaded on the cloud that are needed for many of these pipelines, as well as detailed instructions for creating your own
- Links to useful resources for analysis, cloud computing, or running workflows on other providers such as Terra or DNAnexus

https://github.com/wustl-oncology

Malachi Griffith

An end-to-end tutorial that runs immuno.wdl on public data

Running the WASHU Immunogenomics Workflow on Google Cloud - compute1 version

Preamble

This tutorial demonstrates how to run the WASHU immunogenomics pipeline (immuno.wdl) on Google Cloud. This will be done by first setting up the workflow definitions, input data and reference files and a YAML config file on a user's local system. The user will also set up a Google Cloud environment and all the inputs will be staged to this cloud environment. Next Cromwell will be used to execute the pipeline using the specified input and reference files, and finally the results will be pulled back to the local system and cloud resources will be cleaned up.

This version assumes that your are staging your input data files from the WASHU RIS storage1/compute1 cluster. Some steps are run within docker containers on that cluster. It therefore requires that you have access to storage1 disk and ability to launch jobs on compute1 using LSF (bsub).

Source of instructions

This tutorial is a specific example of how to run a specific pipeline (immuno) on a specific example dataset (HCC1395 Tumor/normal cell line pair). The steps below are taken from the following link where you will find a more generic set of documentation that explains in detail how to run any WDL pipeline on the Google Cloud using tools created to assist this process. https://github.com/wustl-oncology/cloud-workflows/tree/main/manual-workflows

Prerequisites

- google-cloud-sdk
- git

Security

https://github.com/wustl-oncology/immuno_gcp_wdl_compute1

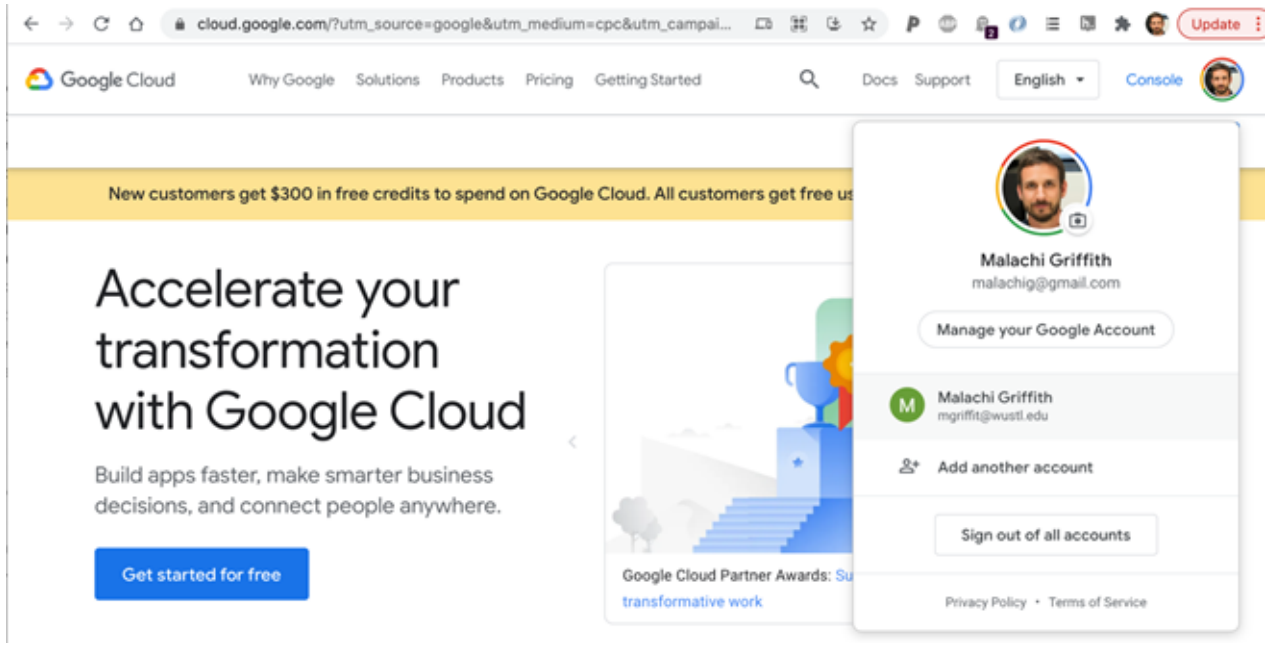
Malachi Griffith

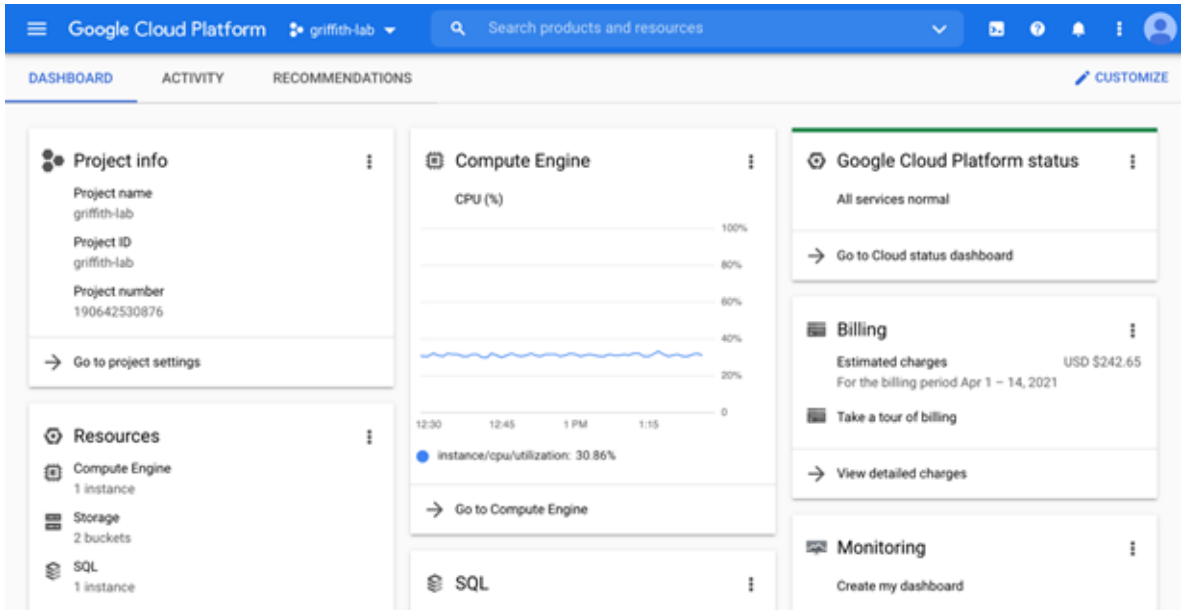
This week's assignment

Running a small workflow, using Google cloud console

Logging into GCP (https://cloud.google.com/)

- Your WASHU email will function as a Google account
  - A very limited functionality Google account associated with your WASHU login
  - You now have two Google accounts. If you use the Google ecosystem a lot, this will be annoying.
  - Better functionality would happen with Google Workspace (formerly known as "G Suite") integration.
  - Use Google Chrome Profiles to keep the two accounts separate





Malachi Griffith

This week's assignment

Running a small workflow, using Google cloud console